

Multi-Label Emotion Classification for Digital Journaling Applications

- Aakriti Dhakal



Problem Statement

- Digital journaling market growing rapidly (\$6.68B → \$15.1B)
- Most apps lack intelligent emotion analysis
- No personalized wellness insights

The Solution: AI-powered journaling with real-time emotion detection

- User writes → BERT analyzes → Detects emotions → Personalized recommendations

System Architecture

React Native App (iOS)



User writes journal entry



ML Model: BERT (110M parameters)



Detects: Joy, Sadness, Anger, Fear, Neutral



Generates personalized wellness tips

Model Architecture

Input Text (max 48 tokens)



BERT Encoder (110M parameters, 12 layers)



Dropout (0.11)



Linear Classifier (768 → 5 emotions)



Multi-label Predictions

Hyperparameter Optimization

Trial	Val Loss	Learning Rate	Dropout	Batch
1	0.1925	1.15e-05	0.11	32
0	0.1988	1.08e-05	0.27	16
2	0.2146	2.91e-05	0.24	32

- **Optuna Search Results (3 trials):**
- **Impact:** Baseline 72% → Optimized 88% (+16% improvement)

Training Results

- **Performance Across Datasets:**
- **Test Metrics:** Precision 54.29% | Recall 28.25% | F1 36.47%
- **Analysis:** 27.6% gap indicates overfitting to validation set

Dataset	Accuracy	Loss
Training	94.1%	0.113
Validation	92.65%	0.170
Test	65.06%	0.201

Model Training Demo

 Copy of SentimentAnalysis.ipynb ☆ ☁

PRO File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

✓ RAM Disk



Multi-Label Emotion Classification with BERT and PyTorch Lightning

In this advanced deep learning project, I developed a multi-label emotion classification model using the GoEmotions dataset. The task involved detecting multiple emotions expressed within a single text sample. To address this, I fine-tuned a pre-trained BERT model rather than training from scratch, utilizing the power of transfer learning for natural language understanding.

The implementation utilized PyTorch Lightning to streamline model training, Weights & Biases (W&B) for performance logging and visualization, and Optuna for systematic hyperparameter optimization. The project emphasized efficient data preprocessing, robust training with checkpointing and early stopping, and detailed evaluation using key metrics including accuracy, precision, recall, and F1-score.

The steps of how we're doing model training is:

- Step 1: Data selection and Preprocessing -
- Step 2: Model Selection & Implementation
- Step 3: Model Training
- Step 4: Reporting
- Step 5: Hyperparameter Tuning
- Step 6: Model Evaluation
- Step 7: Reflection

[2] ✓ 14s



```
1 # Install necessary packages for the project. -q flag makes pip install quietly
  # without giving verbose output
2
3 !pip install datasets torch torchvision pytorch-lightning transformers datasets
  optuna wandb scikit-learn -q
```

{ } Variables

 Terminal



✓ 7:21 PM

 T4 (Python 3)

Key Results & Analysis

Strengths:

- Exceeds baseline (20% random → 65% actual)
- High precision (54%) - reliable predictions
- Successfully handles multi-label emotions

Challenges:

- Overfitting (27.6% validation-test gap)
- Low recall (28%) - conservative predictions
- Domain mismatch (Reddit vs journals)

Limitation

Technical Limitations:

1. Overfitting to validation set
2. Low recall on rare emotions
3. Large model size (437MB)
4. Domain adaptation needed

Future Improvements

- Reduce overfitting
- Data augmentation for rare emotions
- Model compression
- Fine-tune on real journal entries
- Expand to 28 emotions
- Personalized user models

Q&A

- Ask me whatever questions you may have

ThankYou

